

3 – CONTROL STRUCTURES & SEQUENTIAL & CONDITIONALS

Control structure are fundamental programming constructs that dictates how the lines of code is processed. They allow you to control the flow of your program rather than having it simply run from top to bottom in a straight line. Understanding control structures allows you to predict the result of a set of statements. The following are the main control structures:

Sequence/Sequential: The statements are processed in order no skips or jumps.

Conditional/Branching: Some statement or blocks of statements are sometimes processed or sometimes ignored.

Repetition/Looping/Iteration: Statements or blocks of statements may be processed zero or more times

Function/Method/Procedure: Statements or blocks of statements are labeled with a name. The statements are invoked whenever the logic is needed.

Control structures may be nested i.e. you may put a loop in a in statement or put a switch statemen inside of a loop to achieve complex behaviors.

The following keywords alter the normal flow inside of control structures:

break – exits the loop early.

continue – skips to the next iteration.

return – exits a function.

Each control structure brings some benefit(s) to the language.

3.a – Sequential

This is the natural order of processing; you start with the first statement and then go to the next and continue until to get to the end. This is very simplistic but at the same time there are no skips or repeats. There is a guarantee that all the statements will be processed exactly once. It serves as the foundations of most processing techniques.

Because all the statements are processed in the same order this control structure will do the exact thing EVERYTIME it is executed (i.e. behave the same way every time it is processed). So additional control structure is needed to build any substantial program i.e., one that can do at least two or more tasks or to behave differently every time you run it.

```
#set of statements without skips or repeats
>>> name = input('Please enter your name: ')
Please enter your name: Narendra
>>> program = input('Please enter your program: ')
Please enter your program: Game - Programming
>>> print(f'Hello {name} welcome to {program}')
Hello Narendra welcome to Game - Programming
```

The above code will always do the same thing: prompt for a string, reading the string prompt for a second string and read that also. Then it prints a message. Programs written with only sequence control structure is capable of completing only a single task.

3.b – Conditionals

This control structure allows you to process or ignore code blocks depending (true/false blocks). This gives immense power to the language, now your programs are able to complete multiple/different task. Simply implement the individual tasks in different blocks. If there are more task to be implement then nest more conditionals.

Like most other modern languages there is a match statement that does the same job as a switch statement in other languages.

Notice unlike C# there is no parenthesis around the assertion nor are there curly braces to delimit the true or the else code blocks. Blocks are defined by indentations. Most IDE's will manage indentations for you (VS Code use 4 spaces for an indent), but if you paste code from another source, then there could be problems. Visually you may not be able to discern between 4 spaces and a tab, the python interpreter will complain most vehemently.

3.b.1 – if

This is the basic syntax to test if a condition is true. If so, the indented code will be executed, if not it will be skipped.

```
n = 5
if n > 2:
    print('Larger than 2')
```

3.b.2 – else

This is a way to specify some code that will be executed if a condition is false.

```
age = 18
if age > 16:
    print('Can drive')
else:
    print('Don't drive')
```

3.b.3 – elif

In addition to the initial If statement condition, you can add extra conditions to test if the first condition is false.

Once an elif condition is true, the rest of the elif conditions are no longer checked and are skipped.

```
weather = 'sunny'
if weather == 'rain':
    print('bring umbrella')
elif weather == 'sunny':
    print('bring sunglasses')
elif weather == 'snow':
    print('bring gloves')
```

3.b.4 – and

This expects the conditions on both sides of the and to be true.

```
s = 58
if s < 60 and s > 50:
    print('Your grade is C')
```

3.b.5 – or

This expects either of the conditions on both sides of the or to be true. Basically, both conditions cannot be false.

```
age = 12
if age < 16 or age > 200:
    print("Can't drive")
```

3.b.6 – not

This will flip the original result of the condition. e.g. if it was true then it's now false.

```
if not 3 > 1:
    print('something')
#Will not be printed.
```

3.b.7 – Ternary statements

The *ternary statement* is a short hand if-else statement.

```
#ternary statement
«value if assertion is true» «assertion» «value if assertion is false»:

status = 'Wealthy' if asset > 1_000_000 else 'Normal'
```

There is a short hand version

```
#ternary statement
>>> a
345
>>> a or 'Hello'
345
>>> a and 'Hello'
'Hello'
```

3.c – Comparison Operators

These comparison (relational) operators allow you to compare two similar typed arguments and either return True or False. They work the same way as in other languages.

3.c.1 – Greater than (>) operator

This operator will return True if the left side is greater than the right side otherwise it will return False.

```
>>> -2 > 3
False

>>> 1 > 1
False

>>> 3 > 2
True

>>> 1.2 > 1.4
False

>>> 3.1 > 3.1
False

>>> 9.8 > 7.2
True

>>> False > True
False

>>> True > True
False

>>> True > False
```

```
True

>>> 'apple' > 'zebra'
False

>>> 'zebra' > 'apple'
True

>>> 'baby' > 'baby'
False

>>> [4, 9] > [5, 0]
False

>>> [2, 3] > [2, 3]
False

>>> [5] > [4]
True
```

3.c.2 – Less than (<) operator

This operator will return True if the left side is less than the right side otherwise it will return False.

```
>>> -2 < 3
True

>>> 1 < 1
False

>>> 1.2 < 1.4
True

>>> False < True
True
```

```
>>> 'apple' < 'zebra'
True

>>> 'baby' < 'baby'
False

>>> [4, 9] < [5, 0]
True
```

3.c.3 – Greater than or equal (>=) operator

This operator will return True if the left side is greater than or equal to the right side otherwise it will return False.

```
>>> 3 >= 2
True

>>> 1.2 > 1.4
False

>>> 3.1 >= 3.1
True

>>> 9.8 > 7.2
True
```

3.c.4 – Less than or equal (<=) operator

This operator will return True if the left side is less than the or equal right side otherwise it will return False.

```
>>> -2 <= 3
True

>>> 1.2 <= 1.4
False
```

```
>>> False <= True
True

>>> 'apple' <= 'zebra'
True

>>> [4, 9] <= [5, 0]
True
```

3.c.5 – Equality (==) operator

This operator will return True only if the left side is the same as the right side otherwise it will return False.

```
>>> 1 == 1
True

>>> 3.1 == 3.1
True

>>> 9.8 == 7.2
False

>>> False == False
True

>>> True == True
True

>>> 'baby' == 'baby'
True

>>> [2, 3] == [2, 3]
True
```

3.c.6 – Not equals (!=) operator

This operator will return True if the left side is greater than the right side otherwise it will return False.

```
>>> -2 != 3
True

>>> 1 != 1
False

>>> 9.8 != 7.2
True

>>> True != False
True

>>> 'zebra' != 'apple'
True

>>> 'baby' != 'baby'
False

>>> [5] != [4]
True
```

3.d – Logic Operators

These operators allow you to implement more complex logic. These operators return a bool value. The “and” and the “or” operators takes two arguments while the “not” operator takes a single expression.

Fun fact

These operators behave exactly as in other languages.

3.d.1 – Logical and operator

The expression “x and y” first evaluates *x*; if *x* is false, its value is returned; otherwise, *y* is evaluated and the resulting value is returned.

```
# and returns true if both expressions are true, false otherwise

>>> age = 20
>>> province = "QC"

>>> age >= 18 and province == "ON"
False
```

3.d.2 – Logical or operator

The expression "x or y" first evaluates *x*; if *x* is true, its value is returned; otherwise, *y* is evaluated and the resulting value is returned.

```
# or returns true if either expression is true, false otherwise

>>> age = 50
>>> service = 6

>>> age > 65 or service > 10
False
```

3.d.3 – Logical not operator

The operator "not" yields "True" if its argument is false, "False" otherwise. Thus, this operator takes a single argument and return either True or False.

```
# not flips the bool value

>>> age = 50

>>> not age > 65
True
```

3.e – Bitwise Operators

The bitwise operators are (&, |, ^, ~, << and >>) These operators take two integers and returns an integer. It treats numbers as a sequence of binary digits (bits) and perform operations bit by bit.

To get a better understanding binary values are used in the demonstration.

Please be advised these operators will only work with operands of the int type.

3.e.1 – Bitwise AND (&) Operator

The bitwise AND operator compares each bit of the first operand to the corresponding bit of the second operand. If both bits are 1, the corresponding result bit is set to 1. Otherwise, the corresponding result bit is set to 0.

```
>>> 9 & 3      # and operator
1              # 1001 & 0011 = 0001
>>> 15 & 3
3              # 1111 & 0011 = 0011
```

3.e.2 – Bitwise OR (|) Operator

The bitwise OR operator compares each bit of the first operand to the corresponding bit of the second operand. If either bit is 1, the corresponding result bit is set to 1. Otherwise, the corresponding result bit is set to 0.

```
>>> 9 | 3      # or operator
11             # 1001 | 0011 = 1011
>>> 15 | 3
15             # 1111 | 0011 = 1111
```

3.e.3 – Bitwise XOR (^) Operator

The bitwise XOR operator compares each bit of the first operand to the corresponding bit of the second operand. If the bits are not the same, the corresponding result bit is set to 1. Otherwise, the corresponding result bit is set to 0.

```
>>> 9 ^ 3      # xor operator
```

```
10          # 1001 ^ 0011 = 1010
>>> 15 ^ 3
12          # 1111 ^ 0011 = 1100
```

3.e.4 – Bitwise NOT (~) Operator

The bitwise NOT operator is a unary operator that flips the bits of its operand..

```
>>> ~3      # not operator
-4          # ~0 11 = 0001
```

3.e.5 – Left Shift (<<) Operator

The left shift operator shifts the bits of the number to the left and fills 0 on voids left as a result.

```
>>> 9 << 1   # left shift operator
18          # 1001 * 2 = 10010
>>> 9 << 2
36          # 1001 * 4 = 100100
```

3.e.6 – Right Shift (>>) Operator

The right shift operator shifts the bits of the number to the right and fills 0 on voids left as a result.

```
>>> 9 >> 1   # right shift operator
4           # 1001 / 2 = 100
>>> 9 >> 2
2           # 1001 / 4 = 10
```

Best Practice

- Use bitwise operators sparingly and only when necessary
- Avoid using bitwise operators to perform arithmetic operations, as this can make your code harder to understand.

x.b.3 – Match statement

The `if` statement allows only two-way branching. In the situation where multi-way branching is required, then the match statement can be used.

This was introduced in Python 3.10 and is similar in purpose and working as the switch statement in most other languages.

```
day = input('Enter the three letter day of the week: ')

match(day.lower()):
    #make it lowercase
    case 'sat' | 'sun':
        #two cases combined using
        print(f'{day} is a family day')
    case _:
        #this will case the remaining cases
        print(f'{day} is a work day')
```

```
x = int(input('Enter a number between 1 and 20: '))

match x:
    case 1:
        print('one')
    case 2 | 3:
        print('two or three')
    case x if x % 5 == 0:
        print('multiple of 5')
    case x if x > 16:
        print('greater than 16')
    case _:
        print('Value not in range')
```

```
name = input('Enter all your names: ')
names = name.split()
match x:
    case [x, y]:
        print(f'two names {x} {y}')
    case [x, y, z]:
        print(f'three names {x} {y} {z}')
    case [x, *y, z]:
        print(f'more than three names {x} {' '.join(y)} {z}')
    case _:
        print('Unkown error')
```

This match statement is more readable than the corresponding if-statement and in some (advanced) cases it is more powerful.